

Soumyadeep Nath

Aspiring Data Scientist · Full-Stack AI Builder · Forward Deployed Specialist

"Literature taught me to think clearly and communicate well. Data Science gives me analytical and technical leverage. Together, they let me build systems and explain them."

soumyadeep9300@gmail.com | github.com/sdn9300 | sdn9300.github.io

Kolkata, West Bengal, India | IIT Roorkee Executive PG — Data Science & AI



Table of Contents

1. Identity & Core Positioning
 2. Professional Profile Snapshot
 3. Technical Profile
 4. Flagship Projects
 - 4.5. Python Learning Progress - Adaptive Tutoring Record
 5. Workflow Preferences & Engineering Principles
 6. Communication & Writing Style
 7. Design & Portfolio Preferences
 8. Goals, Learning Priorities & Current Horizon
 9. Practical Constraints and Context
 10. Observed Behavioral Patterns Across Conversations
 11. Resume Engineering & Job Search Status
 12. How to Use This Document
 13. Living Notes
- Appendix A - Quick Identity Card
- Appendix B - Active Learning Frameworks Full Spec

A living, consolidated reference document - merged from About_Me.md, about_me_updated.md, and the Technical Profile & Agentic Workflow Specification (PDF) - for reuse across future projects, chats, and AI collaborators.

1. Identity & Core Positioning

Legal Name: Soumyadeep Nath

Geographic Base: Kolkata, West Bengal, India

Contact: soumyadeep9300@gmail.com github.com/sdn9300 sdn9300.github.io

Current Education: Executive PG Programme in Data Science & AI, IIT Roorkee

Academic Credentials: M.A. and B.A. (Honours) in English Literature, Presidency University, Kolkata

Current Track: Aspiring Data Scientist -> AI Engineer -> eventually Forward Deployed Engineer (FDE)-style roles

Target Roles (near-term): AI/ML internships, entry-level Data Science roles

Core Role Framing: Aspiring Data Scientist, Full-Stack AI Builder/Engineer, and Forward Deployed Specialist (long-term)

The defining differentiator

A background in English Literature combined with a deliberate pivot into Data Science & AI. This is **not** treated as a liability to minimize - it is actively foregrounded as a genuine professional edge: the ability to **translate technical findings and complex statistical/algorithmic work into language non-technical stakeholders can act on**. Any positioning material (resume, LinkedIn, portfolio bio) should lean into this Literature -> Data Science arc rather than hide it.

Typical framing to reuse: *"Literature taught communication and critical thinking. Data Science provides analytical and technical leverage. The combination creates a professional capable of both building systems and explaining them."*

Why the literary background transfers technically

The causal relationship between literary training and quantitative analysis is deeper than communication skills alone. The study of literature demands parsing complex, highly unstructured semantic networks, identifying latent thematic patterns, and establishing structural rules governing textual systems. When translated into technical domains, this cognitive training yields a natural aptitude for:

- **Advanced feature engineering** - recognizing meaningful signals within noisy, unstructured inputs
- **Taxonomy-driven extraction** - designing classification and extraction systems (as seen in Future Fit's regex taxonomy engine)
- **Strict semantic validation** - treating schema contracts and spec boundaries as meaning-preserving constraints
- **Narrative cohesion in system design** - viewing code not as a sequence of instructions but as a structured linguistic system that must possess logical and narrative coherence

This explains the deep-seated preference for Spec-Driven Development: software specifications are treated as semantic contracts governing automated execution, in the same way literary theory treats structural rules as governing textual interpretation.

Credential honesty principle

Accurate, consistent framing is preferred over inflated claims, titles, or exaggerated scope. **When in doubt, undersell rather than oversell.** This principle extends into engineering choices too - e.g., building strict guardrails to prevent LLM hallucination in his own products mirrors this same value applied to software.

Background that matters for positioning

- M.A. & B.A. (Honours), English Literature, Presidency University, Kolkata
- Strong language, editing, reading, and communication skills
- Over a year of private tutoring experience (school-level English), involving data-driven student assessment
- Still early in the technical journey - accuracy matters more than sounding advanced

2. Professional Profile Snapshot

Attribute	Detail
Role framing	Aspiring Data Scientist / Full-Stack AI Builder / Forward Deployed Specialist (long-term)
Target roles	AI/ML internships, entry-level Data Science roles; long-term: AI Engineering, FDE
Core strength	Bridging technical execution and stakeholder communication (interdisciplinary translation)
Working philosophy	Spec-Driven Development (SDD) - specification and architecture before code
Portfolio anchors	AlignResume, Future Fit, Altitude & Heat Adaptation in FIFA World Cups, Gleaner/The Harvester, Overture
Primary core strengths	Interdisciplinary translation, spec-driven engineering, robust inference

3. Technical Profile

Current skill snapshot (self-assessed, as of June 2026)

Skill Area	Level	Notes
Python	Basic to early practical	Active learner; pandas/NumPy in progress via adaptive tutoring
SQL	Beginner	Foundational queries, joins, aggregations
Statistics	Descriptive + foundational inference	Hypothesis testing, A/B testing covered in coursework
R	Actively learning	Primary statistical modelling tool; GLMM, visualisation
Machine Learning	Foundational (in progress)	Regression, classification, tree-based ensembles via IIT Roorkee programme
GenAI / LLMs	Practical interest	Structured prompts, guardrails, RAG, workflow integration - strongest applied area

Current skill-level snapshot (self-assessed, as of latest update)

- **Python:** basic to early practical level (data ingestion pipelines, scripting, automation)
- **SQL:** beginner
- **Statistics:** descriptive statistics and foundational inference work; advancing into GLMM/regression via R
- **R:** learning, with the goal of becoming proficient for statistical analysis, modeling, and reproducible research - currently the primary engine for the flagship sports analytics project
- **Machine Learning:** building fundamentals
- **GenAI/LLMs:** practical, applied interest - especially structured prompts, hallucination guardrails, and GenAI workflows for data science/product use cases

Important clarification - this is not a discrepancy, it's a deliberate strategy: Soumyadeep's personal skill level is genuinely "basic to early practical" across Python, SQL, and ML. However, he deliberately *chooses* to undertake advanced, production-style projects - GLMM/Negative Binomial modeling, full CI/CD, Docker, deterministic LLM guardrails - by working **with AI assistance** to architect and build at a level beyond his current solo skill level. This is the explicit mechanism behind Overture, Gleaner, AlignResume, Future Fit, and the Altitude & Heat Adaptation project: AI-assisted construction of production-grade systems as a *learning vehicle* and portfolio strategy, not a misrepresentation of personal ability. When describing these projects (resumes, portfolio, READMEs), the **project's architecture and rigor can be described at its true production-grade level**, while personal skill claims should stay honest and self-assessed per the credential-honesty principle. The two are not in conflict - the projects demonstrate capability built through AI-assisted, spec-driven engineering, and that *is* a legitimate, describable skill in itself.

Languages & Frameworks

- **R** - primary computational engine for statistical analysis, hypothesis testing, GLMM, Negative Binomial regression, data visualization
- **Python** - data engineering, web scraping, API consumption, pipeline automation, general scripting
- **Next.js 14 (App Router), TypeScript, Tailwind CSS** - user-facing/product front ends

- Vanilla HTML/CSS/JavaScript, Vite

Data & ML Tooling

- Groq API (structured LLM outputs, deterministic hallucination guardrails)
- Prophet (time-series forecasting)
- Apriori / association rule mining
- Pandas, Streamlit, Plotly
- StatsBomb API, FBref parsers, BeautifulSoup (sports data sourcing/scraping)
- OpenStreetMap (OSM), Open-Elevation API, Meteostat API (geospatial & weather data)
- Firecrawl API, Google Sheets API

Engineering & DevOps Tooling

- Git and GitHub, GitHub Actions (CI/CD)
- Docker (containerization)
- pytest (test hardening)
- Redis (async queue for Overture pipeline)
- Prometheus (metrics instrumentation for Overture)
- SQLite (run history storage for Overture dashboard)
- WakaTime (coding-time tracking)
- ReportLab (PDF generation)

Deployment

- Vercel (Edge Runtime), GitHub Pages, Streamlit Community Cloud

Profile/Branding Tools

- shields.io ([for - the - badge](#) style), readme-typing-svg, capsule-render, GitHub stats cards

Reference materials currently in active use

These are the specific materials Soumyadeep is working from during the current Python learning phase. Any AI assistant or collaborator helping with Python/data-analysis topics should calibrate explanations to these resources and use consistent terminology.

Resource	Type	Primary Use
<i>Python for Data Analysis</i> - Wes McKinney (2nd Ed.)	Textbook	Primary pandas + NumPy structured learning
NumPy Cheat Sheet (GeeksForGeeks-style)	Quick reference	Array creation, vectorized math, slicing, reshaping

Resource	Type	Primary Use
Pandas Cheat Sheet (official pandas.pydata.org / Irv Lustig)	Quick reference	DataFrame operations, reshaping, merging, filtering
Python Beginner's Cheat Sheet (<i>Python Crash Course</i>)	Quick reference	Core Python syntax, lists, dicts, loops, functions
Google Colab / Jupyter Notebook	Coding environment	All Python experimentation and problem-solving

Technical preferences

- Reproducibility matters
- Clean pipelines matter
- Documentation matters
- Testing matters
- Strong naming and modular design matter
- A project should be understandable from its README and structure, not only from the code

4. Flagship Projects

AlignResume (a.k.a. "Resume Shapeshifter")

An AI-powered resume optimization platform built on **Next.js 14**, **TypeScript**, and the **Groq API**, focused on matching resumes to job descriptions, match scoring, gap analysis, bullet rewriting, and tailoring workflows. Its centerpiece engineering feature is a **deterministic hallucination guardrail** - programmatically anchoring LLM resume optimization to validated user input data, blocking the generation of fabricated credentials. Paired with a side-by-side PDF "proof artifact" (built via Playwright/React-PDF) as the key deliverable. Built with Zod schemas and a modular `lib/` and `prompts/` structure; domain models include `ResumeProfile`, `JobDescriptionProfile`, and `TailoringRun`. Deployed at align-resume-beta.vercel.app.

- **MVP non-goals:** auto-apply, large-scale scraping, cover letters (initially), ATS guarantees.
- Values: truthfulness guardrails, explainable match scores, side-by-side diff review, polished PDF export as proof.

Future Fit

A **skill trend intelligence platform** processing **224,605 skill mentions** extracted from **50,000+ global AI/Data Science job postings (2020-2026)**, via a taxonomy-driven regex extraction engine with word-boundary matching across 100+ skills. A 10-step deterministic cleaning pipeline feeds two parallel analytical engines: a **Market Basket Analysis (Apriori / mlxtend)** engine mapping **230 frequent skill itemsets** and extracting association rules (lift ≥ 0.90 , confidence ≥ 0.38) to surface co-occurrence trends (e.g., PyTorch $\rightarrow$ TensorFlow); and a **Prophet**-based tiered yearly forecasting model projecting demand shares into 2027, with a dynamic fallback to linear trend extrapolation when the 80% confidence interval width exceeds a 1.5x variance threshold. Deployed as a **6-panel Streamlit dashboard** with year/category filters,

interactive Plotly visualizations, and a **Groq LLM-powered (LLaMA 3.3 70B) Skill Gap Advisor** that generates contextual learning-path recommendations enriched with the mined association-rule insights. Live: skilltrendanalysis-rf3zefgsjaa4l9f8pu2prb.streamlit.app Repo: github.com/sdn9300/Future-Fit-AI-Powered-Skill-Trend-Analysis

Altitude & Heat Adaptation in FIFA World Cups *(current flagship, in active development)*

A sports analytics research project investigating whether **altitude, heat, humidity, and travel fatigue measurably degrade football team performance**, using historical FIFA World Cup data, with forward-looking range-based projections for the **2026 FIFA World Cup**.

- **Data ingestion:** multi-tiered Python pipeline harvesting granular match metrics from **StatsBomb** and **FBref**, combined with environmental parameters from **Meteostat**, **Open-Elevation API**, and geocoded **OpenStreetMap** coordinates to compute historical weather conditions and travel demands.
- **Core statistical model:** Generalized Linear Mixed Model (GLMM) with a Negative Binomial distribution, fit in **R**, to handle overdispersed count data representing team performance markers:

$$\log(\mu_{ij}) = \beta_0 + \beta_1 \text{Altitude}_{ei} + \beta_2 \text{ApparentTemperature}_{ej} + \beta_3 \text{TravelDistance}_{ej} + z_{bj}$$

where μ_{ij} is the expected performance outcome of team i in match j , modeled relative to fixed environmental covariates plus a random intercept vector bj capturing team-specific variance.

- **Key architectural decision:** Predictive modeling is deliberately subordinated to inferential/hypothesis-testing modeling. Because ground truth for future tournaments is inherently limited, 2026 outputs are presented as **conditional ranges, not point estimates**, using **leave-one-tournament-out cross-validation** - framed consistently as "projections under historical patterns," never as confident predictions. (*General principle worth reapplying elsewhere: when ground truth is limited, subordinate prediction to inference, and present forecasts as conditional ranges.*)
- **Planning artifacts:** formal hypotheses H1-H5; a 10-phase R data pipeline master plan; an Architecture Design Document with ADRs; a Mission Plan with a risk register and Go/No-Go execution gates (military/operational-style planning); an Evaluation Plan with weighted scoring rubrics; and a 37-scenario Edge Case Registry across six categories.
- **Companion site:** a Next.js 14 portfolio/dashboard site (dark, high-contrast, sports-analytics aesthetic) to display statistical visualizations and range projections, including TypeScript interfaces and venue JSON data.
- **Status:** planning is complete; the R pipeline build phase is next, followed by the companion site deployment.

Gleaner

A multi-board job scraping and ETL pipeline, architected around the **Adapter Design Pattern** to concurrently fetch, normalize, and aggregate listings from **4+ platforms** (Naukri, Indeed, RemoteOK, Wellfound/WWR). Scraping resilience is handled via stealth configuration headers, a custom bot user-agent, and the **Firecrawl API's** structured extraction logic for anti-bot-resistant retrieval. Cleaning stages apply exact and similarity-based deduplication, role-relevance keyword analysis, and location-based filtering on raw unstructured payloads. Output is dual-channel: local CSV plus live sync to Google Sheets via [gspread/google-auth](#). Execution is controlled through a configurable CLI driven by centralized YAML files (logging levels, output size limits, targeted source runs). Reflects the practical-pipelines, modular-systems, workflow-automation interest. Repo: github.com/sdn9300/gleaner-job-scout. Currently being evaluated for

potential integration as a feature inside AlignResume.

Overture Outreach - AI-Powered Cold Email Outreach Platform

A production-hardened, full-stack automated outreach engineering platform executing secure pipeline delivery and performance tracking. Originally developed as a sprint-based exercise but evolved into a complete, multi-component system.

Core Architecture: Modular backend outreach engine built in Python, orchestrating a dual-stage execution pipeline:

- Stage 1 - **Deterministic HTML formatting** (structured, reproducible output; factual grounding enforced before any LLM touches the content)
- Stage 2 - **Dynamic tone polishing via Groq API (LLaMA 3.3 70B)** (optional LLM enhancement applied only after deterministic structure is locked)

Authentication & Security: Secure Google OAuth2 flow using Google Client APIs to obtain and store authorization tokens (`token.json`), enabling programmatic, rate-safe cold mail delivery directly through the native Gmail API.

Async Pipeline & Telemetry:

- Asynchronous background execution via a **Redis queue system**
- Instrumentation via a **Prometheus Metrics Server** (exposed at <http://localhost:8000>)
- Analytics counters tracking active mailings, send rates, and delivery outcomes

Presentation Dashboard: Responsive **Next.js** frontend UI dashboard built in **TypeScript** and **Tailwind CSS**, deployed on Vercel at frontend-three-azure-97.vercel.app, displaying:

- **SQLite**-backed run histories
- Metrics and instrumentation panels
- Contact table with filters
- Live mail preview and composition interface

Hardening: Comprehensive **pytest** test suite; Docker containerization for isolated, reproducible execution.

Veritas Daily Reflection Journal

Personal / Open-source project - 2024 to Present

Full-stack Mobile & Backend Application

A **local-first journaling platform** that lets users capture daily reflections, track moods, build habits, log gratitude, and create time-capsules. Voice input and AI-driven insights (weekly summaries, year-review, playlists) are delivered through an Express backend powered by an OpenAI-compatible LLM. The entire codebase lives in a **single pnpm monorepo** and is developed entirely on Replit.

GitHub: github.com/sdn9300/Veritas-Daily-Reflection-Journal

Responsibility	Details
Monorepo architecture	Designed and maintained a pnpm-workspace monorepo (pnpm-workspace.yaml) cleanly separating mobile, API server, shared libraries, and a mockup sandbox
Mobile app	Built the Expo Router / React Native app (artifacts/mobile) with a tab-based UI (Today, Journal, Jar, Search, Habits, Insights). Integrated AsyncStorage for offline-first persistence, voice-to-text capture, streak tracking, reminders, and PDF export
Backend services	Implemented an Express 5 API (artifacts/api-server) exposing health checks and AI routes (/insights/*). Integrated OpenAI-compatible endpoints via Groq for insights and transcription, with structured JSON responses
Data layer	Managed PostgreSQL schema with Drizzle ORM (lib/db). Exported typed schema and connection pool for both mobile and server
Shared contracts	Authored an OpenAPI spec (lib/api-spec) and used Orval to generate TypeScript client code (lib/api-client-react) and Zod validation schemas (lib/api-zod). Ensured type-safe communication across the full stack
Type safety & tooling	Adopted TypeScript 5.9, shared tsconfig files, enforced pnpm run typecheck in daily workflow, added linting, pre-commit hooks, and automated code-gen scripts
Rapid prototyping	Created a Vite-based UI sandbox (artifacts/mockup-sandbox) for component experimentation without rebuilding the mobile app
CI/CD & automation	Configured Replit-hosted GitHub Actions to run pnpm install --frozen-lockfile , push DB migrations (pnpm --filter @workspace/db run push), and regenerate API clients on merge
Documentation	Comprehensive README, usage guides, and step-by-step workflow for fresh clones and daily development loops

Tech stack:

- **Languages / Runtime:** TypeScript 5.9, JavaScript (Node v24)
- **Frontend:** Expo Router, React Native, React Query, AsyncStorage, Vite (sandbox)
- **Backend:** Express 5, Pino logger, CORS, OpenAI / Groq LLM API integration
- **Database:** PostgreSQL, Drizzle ORM (type-safe queries, migrations)
- **Schema & API:** OpenAPI, Orval (client generation), Zod (validation)
- **Package management:** pnpm workspaces, pnpm catalog (consistent dependency versions)
- **Development environment:** Replit (cloud IDE, Replit mobile app hosting)
- **Build & CI:** npm scripts, GitHub Actions, post-merge automation, type-checking, linting

Demonstrates end-to-end product ownership - from architecture and full-stack implementation to CI/CD and community documentation - while leveraging modern TypeScript-centric tooling and cloud-native development on Replit.

Project Summary Table

Initiative	Core Stack	Domain	Key Innovation
Altitude & Heat Adaptation	R, Python, Next.js 14, StatsBomb, FBref, Meteostat, Open-Elevation, OSM	Sports analytics, applied climatology, inferential statistics	GLMM Negative Binomial regression; range-based projections with leave-one-tournament-out CV
AlignResume	Next.js 14, TypeScript, Groq API, Zod, Playwright/React-PDF	LLM guardrails & semantic document optimization	Deterministic guardrails preventing fabricated/unverified credentials
Future Fit	Python, Prophet, mlxtend (Apriori), SciPy, Streamlit, Plotly, Groq API	Labor market skill trend forecasting & association mining	224K+ skill mentions mined from 50K+ postings; Prophet forecasting + Apriori association rules + LLM-powered Skill Gap Advisor
Gleaner	Python, BeautifulSoup4, Firecrawl API, Google Sheets API, YAML	Job-listing scraping & ETL	Adapter-pattern multi-source scraper (4+ boards) with dedup, keyword/location filtering, and YAML-driven CLI
Overture Outreach	Next.js, TypeScript, Python, Redis, Prometheus, SQLite, Groq API, Gmail API	Automated cold email outreach pipeline & telemetry	Dual-stage formatting engine (deterministic HTML to LLM polishing via Groq); full Prometheus analytics instrumentation; OAuth2 Gmail delivery
Veritas Daily Reflection Journal	TypeScript 5.9, Expo Router, React Native, Express 5, PostgreSQL, Drizzle ORM, Groq API, Zod, Orval, pnpm workspaces	Full-stack mobile journaling platform with AI insights	pnpm monorepo separating mobile / API / shared libs; Orval-generated type-safe API clients; offline-first with voice-to-text, streak tracking, and time-capsules

Broader project philosophy

Prefers projects that:

- solve real problems, using real datasets and real-world constraints (not tutorial clones)
- feel employable and portfolio-worthy
- demonstrate full end-to-end ownership (data collection -> cleaning -> analysis -> modeling -> deployment -> documentation -> evaluation)
- maximize signal-to-effort ratio by combining multiple capabilities at once (data engineering + forecasting + analytics + AI integration, etc.)

Portfolio principles

- Show concrete outcomes - every project should have a visible result (deployed URL, verifiable metric, documented finding)

- Favor deployed work over local-only notebooks
- Prefer real data over toy examples
- Build systems, not isolated demos - each project should demonstrate architectural thinking, not just feature execution
- Make the work easy to inspect and understand from the README alone

4.5. Python Learning Progress - Adaptive Tutoring Record

This section documents Soumyadeep's **actual, session-verified learning progress** in Python for data analysis. It exists because the rest of §3 and §4 describe *project-level* work (much of it AI-assisted); this section captures what has been *personally demonstrated and verified through hands-on problem-solving*. These are not aspirational claims - they are observable, tested results from structured adaptive tutoring sessions.

Learning format used

Soumyadeep uses an **Adaptive Coding Examination System** - a structured, 5-level difficulty ladder with the following mechanics:

Level	Label	Adaptation rule
1	Easy	Baseline starting point
2	Easy-Medium	+ Unlock after Level 1 correct
3	Medium	+ Unlock after Level 2 correct
4	Medium-Hard	+ Unlock after Level 3 correct
5	Hard	+ Unlock after Level 4 correct

After any failed attempt: drop one level. After a partial attempt: hold level. This ensures Soumyadeep is always working at the productive edge of competence rather than coasting or being overwhelmed.

Difficulty levels reached

Level	Label	Status
Level 1	Easy	+ Completed
Level 2	Easy-Medium	+ Completed - **Perfect Score** on "The Sales Report"
Level 3	Medium	~ In Progress - "The Student Grade Analyzer"
Level 4	Medium-Hard	Upcoming
Level 5	Hard	Upcoming

Problem record

DONE "The Sales Report" - Level 2 (Easy-Medium) - Perfect Score

Concepts tested and confirmed mastered:

Skill	Verified
<code>groupby()</code> aggregations	+
<code>idxmax()</code> - finding the index of the max in a grouped Series	+
<code>df.loc[]</code> - label-based selection using a computed index	+
<code>np.select()</code> - multi-condition column creation	+

Notable performance observation: Soumyadeep wrote **explicit compound boolean conditions** (e.g., `(df['col_A'] > x) & (df['col_B'] == y)`) where a simpler, minimal single-condition approach would have been sufficient and more idiomatic. This reflects **stronger-than-expected logical precision for a stated beginner** - deliberate reasoning over minimal coding. This pattern has been consistent.

WIP "The Student Grade Analyzer" - Level 3 (Medium) - In Progress

New concepts introduced at Level 3:

Skill	Status
<code>.round(n)</code> - rounding float columns to n decimal places	Introduced
Boolean row filtering - selecting rows by multi-condition masks	Introduced
<code>.value_counts()</code> - frequency counting on a Series	Introduced

NumPy Comprehensive Lab - 15 Questions

A full 15-question lab was assigned covering all foundational NumPy topics. Status: assigned; full evaluation pending.

Topics the lab covers:

Topic	Subtopics
Array creation	<code>np.array()</code> , <code>np.zeros()</code> , <code>np.ones()</code> , <code>np.arange()</code> , <code>np.linspace()</code> , <code>np.random.rand()</code> , <code>np.random.randint()</code> , <code>np.random.randn()</code>
Array properties	<code>.shape</code> , <code>.dtype</code> , <code>.ndim</code> , <code>.size</code>
Indexing	1D indexing, 2D <code>[row, col]</code> indexing, negative indexing
Slicing	<code>arr[start:stop:step]</code> , 2D row/column extraction, submatrix slicing
Vectorized math	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>**</code> , <code>np.sqrt()</code> , <code>np.exp()</code> , <code>np.log()</code>
Statistical methods	<code>np.sum()</code> , <code>np.mean()</code> , <code>np.std()</code> , <code>np.min()</code> , <code>np.max()</code> with <code>axis</code> parameter
Broadcasting	Scalar-array and compatible shape operations
Reshaping	<code>.reshape()</code> , <code>.flatten()</code> , <code>.ravel()</code>
Boolean indexing	<code>arr[arr > value]</code> , compound boolean conditions
Universal functions	<code>np.sqrt</code> , <code>np.abs</code> , <code>np.ceil</code> , <code>np.floor</code>

Confirmed pandas patterns in use

These are patterns that have been written correctly in actual problem sessions - not just mentioned:

```
# groupby + aggregation
df.groupby('region')['revenue'].sum()

# Finding index of max within groups
best_idx = df.groupby('region')['revenue'].idxmax()

# Label-based lookup with loc
df.loc[best_idx, 'product']

# Multi-condition column assignment with np.select
import numpy as np
conditions = [
    (df['score'] >= 90),
    (df['score'] >= 75) & (df['score'] < 90),
    (df['score'] < 75)
]
choices = ['A', 'B', 'C']
df['grade'] = np.select(conditions, choices, default='F')

# Rounding floats
df['avg_score'] = df['avg_score'].round(2)

# Boolean row filtering
high_scorers = df[df['score'] >= 85]

# Value counts
df['grade'].value_counts()
```

Preferred interaction format for coding help

This is the exact structured flow Soumyadeep responds best to - consistently requested and effective:

1. Problem presented clearly (statement + constraints + examples)
2. Offer: "Would you like a hint or skeleton code?"
-> Soumyadeep almost always requests hints / skeleton first
3. Provide scaffold: partial code with blanks or guided structure
4. Soumyadeep attempts the solution independently
5. Submits code
6. Structured feedback:
 - DONE What was correct and why
 - What could be improved or was wrong
 - Optimized / reference solution with explanations
 - Time & Space complexity
 - Edge cases to be aware of
7. Difficulty adjusted based on performance
8. Next problem

Do NOT hand the solution immediately. Soumyadeep explicitly values the attempt process. Giving answers before an attempt undermines the learning loop.

Pandas skills - full status map

A complete, session-verified picture of where things stand across all pandas topics. The "Contextual Details" column comes from Doc 1's formal skill registry.

Skill / Concept	Status	Contextual Implementation Details
<code>groupby()</code> + <code>.sum()</code> / <code>.mean()</code> / <code>.count()</code>	+ Mastered	Multi-variable grouping and aggregation functions
<code>idxmax()</code> on a grouped Series	+ Mastered	Isolating peak indexed values across grouped Series
<code>df.loc[]</code> label-based row/column selection	+ Mastered	Multi-axis index querying confirmed
<code>np.select()</code> for multi-condition column creation	+ Mastered	Multi-condition vector assignment with explicit compound conditions
<code>.round(n)</code>	+ Introduced	Element-wise precision rounding
Boolean row filtering (<code>df[df['col'] > val]</code>)	+ Introduced	Row-wise selection using compound logical masks
<code>.value_counts()</code>	+ Introduced	Frequency profiling via Series
<code>.apply()</code> with lambda or named function	Upcoming	Vector mapping with <code>.apply()</code> and anonymous functions
<code>pivot_table()</code> / <code>melt()</code> / <code>stack()</code> / <code>unstack()</code>	Upcoming	Multi-dimensional reshaping operations
<code>merge()</code> / <code>join()</code> / <code>pd.concat()</code>	Upcoming	Explicit tabular joins, concatenations, and alignment

Skill / Concept	Status	Contextual Implementation Details
<code>.fillna()</code> / <code>.dropna()</code>	Upcoming	Missing data handling
<code>pd.cut()</code> / <code>pd.qcut()</code>	Upcoming	Binning and discretization
Time-series indexing (<code>DatetimeIndex</code>)	Upcoming	Datetime object manipulation for time-series parsing (directly relevant to Future Fit)
<code>groupby().transform()</code>	Upcoming	Group-wise column transforms
<code>groupby().apply()</code> with custom function	Upcoming	Advanced groupby patterns
String accessor <code>.str.*</code>	Upcoming	Vectorized string operations

NumPy lab - completion status

Topic block	Lab status
Array creation	+ Covered
Array properties (<code>.shape</code> , <code>.dtype</code> , <code>.ndim</code> , <code>.size</code>)	+ Covered
Indexing & slicing (1D, 2D, negative)	+ Covered
Vectorized math	+ Covered
Statistical methods with <code>axis</code> parameter	+ Covered
Broadcasting	+ Covered
Reshaping (<code>.reshape()</code> , <code>.flatten()</code> , <code>.ravel()</code>)	+ Covered
Boolean indexing	+ Covered
Universal functions (ufuncs)	+ Covered
Full lab score / evaluation	~ Pending - answers submitted, scoring not yet complete

Active learning frameworks in use (June 2026)

Two structured coding-support systems are now in active use alongside the adaptive pandas/NumPy tutoring:

Framework 1 - General Coding Support System (Modes 1-6)

A multi-mode system covering: Explain (MODE 1), Problem Generator (MODE 2), Solution & Walkthrough (MODE 3), Debugger (MODE 4), Practice System (MODE 5), Doubt Solver (MODE 6). Built around six teaching principles: intuition first, step-by-step thinking, multiple approaches, edge case focus, context engineering, and an evaluation/progress-tracking layer.

Framework 2 - Adaptive Coding Examiner (DSA + Python)

A Computerized Adaptive Testing (CAT)-style system for DSA and Python problem-solving covering:

- 5-level difficulty ladder (Easy -> Easy-Medium -> Medium -> Medium-Hard -> Hard)
- Dynamic difficulty adjustment per attempt (correct -> +1, fail -> -1, partial -> hold)
- Pattern coverage: Arrays, Strings, Hashing, Recursion, Dynamic Programming, Graphs
- Skill tracking: strong topics, weak topics, mistake patterns tracked across sessions
- Problem design principles: progressive difficulty, real interview-style, edge-case awareness
- Evaluation per attempt: correctness, time complexity, space complexity, code quality
- Hard rule: solution never revealed before user attempts

These frameworks complement the pandas/NumPy adaptive tutoring system already in use (§4.5 above) and extend Soumyadeep's structured learning into DSA, algorithms, and broader Python programming.

5. Workflow Preferences & Engineering Principles

Spec-Driven Development (SDD) - non-negotiable

Development is grounded in **Spec-Driven Development**, implemented via the open-source **GitHub Spec Kit** framework. Specifications are the source of truth; code is a secondary implementation detail. This separates the "what" (functional requirements) from the "how" (technical implementation), preventing technical debt, architectural drift, and unstructured AI-assisted development.

SDD Operational Lifecycle:

1. **Constitution** - establish guardrails & coding standards ([speckit.constitution](#))
2. **Specify** - define user-facing behavior & acceptance criteria ([speckit.specify](#))
3. **Plan** - establish technical architecture, schemas, ADRs ([speckit.plan](#))
4. **Tasks** - decompose the plan into granular, testable execution blocks ([speckit.tasks](#))
5. **Implement** - automated code generation and feature construction ([speckit.implement](#))
6. **Validate & Replan** - run automated test suites ([pytest](#)) and address deviations

Specification rigor levels:

Level	Objective	Where it lives	When used
Spec-First	Early alignment only; not maintained post-generation	Initial prompt / disposable doc	Rapid, low-complexity prototyping
Spec-Anchored	Specification evolves with the codebase as a living reference	Repository /docs or /specs	Primary operational standard for flagship systems and statistical pipelines

Level	Objective	Where it lives	When used
Spec-as-Source	Code generated directly from spec; humans don't edit code files	Config/DSL file + automated compiler	Isolated, low-criticality automation tasks only (non-deterministic risk otherwise)

Soumyadeep operates primarily in the **spec-anchored** mode: specs are version-controlled repository assets, verified through automated integration tests, not disposable prompt inputs.

Core engineering/working principles (recurring defaults)

- 1. No code before a spec exists.** Discovery -> Design -> Architecture -> Code.
- 2. Documentation explains "why," not just "what."** Every architectural decision carries its rationale (ADRs).
- 3. Predictive modeling is subordinate to inferential modeling** when ground truth is limited - forecasts as ranges with cross-validation, not point estimates.
- 4. Military/operational framing for planning** - mission phases, risk registers, Go/No-Go gates. A genuine preference, not decoration.
- 5. Debugging is root-cause-first.** If an error occurs, the AI/collaborator must isolate the actual underlying cause, explain it in clear technical terms, and update test suites to prevent regressions - never apply quick fixes or surface-level patches.
- 6. Iterative, progressively deeper deliverables.** Comfortable working in layers (problem statement -> architecture -> implementation -> evaluation -> hardening), often via simple "Continue" prompts.
- 7. Elaborate, highly organized documentation is a feature, not a burden.** Long-form specs, ADRs, schema contracts, registries are preferred over terse output.
- 8. Balance spec detail with execution cost** - to avoid over-engineering, the effort spent refining specs should not exceed the cost of fixing the errors it prevents.

How collaborators/AI assistants are expected to operate

- Act less like a chatbot, more like a senior advisor, systems engineer, architect, researcher, and instructor
- Verify design decisions against the specification before writing code
- Start with structure before implementation; explain rationale, not only results
- Treat complex work as a system, not a single answer
- Be direct when something is weak, unclear, or not credible
- Prioritize what is most useful for employability and long-term growth

Preferred workflow pattern

1. Define the problem clearly
2. Identify constraints
3. Design the structure
4. Propose an architecture or plan

5. Implement or rewrite
6. Evaluate critically
7. Refine for clarity, credibility, and impact

Decision-making style

- Thinks in terms of trade-offs and second-order effects (e.g., "what are the consequences of doing this?" rather than just "what should I do?")
- Cares about how a choice affects portfolio, credibility, and long-term trajectory
- Prefers evidence-based recommendations and comparisons over single-option answers
- Wants the "why this matters," not just the "what to do"

6. Communication & Writing Style

Standing preference (explicit)

Replies should be precise, and explained the way a **world-class expert teacher with 50+ years of teaching experience** would explain something - clear enough that nothing is left ambiguous, with reasoning made visible rather than assumed. Don't just give the answer - give the *why*, the trade-off, and the mental model, calibrated to actually teach.

Tonal defaults

- Academic, authoritative, pedagogical, mathematically/logically precise and verified
- Professional and polished, but not robotic
- Confident without exaggeration
- Analytical and direct when evaluating ideas, projects, or career choices
- Strategic when discussing employability, portfolio design, or long-term goals
- Formal, objective, analytical, using precise engineering/statistical terminology

What is preferred

- Clear hierarchy with headings/subheadings; executive summaries before details
- Detailed markdown tables for structured/comparative/taxonomic content; continuous dense prose for qualitative reasoning; LaTeX equations and schema contracts where relevant
- Phase-wise plans, roadmaps, step-by-step execution structures
- Documentation of the "why" behind technical choices (ADRs), explicit trade-off analyses
- Specific wording improvements for resumes, GitHub READMEs, portfolio copy, LinkedIn content
- Information density over conversational filler; precision never sacrificed for brevity
- Iterative refinement: draft -> critique -> rewrite -> tighten -> elevate (common refinement requests: "more precise," "more impactful," "more professional," "more structured")

What to avoid

- Casual, conversational, overly simplified, or hand-waving explanations
- Speculative or vague/hyperbolic claims, marketing language, unverified assertions
- Bulleted qualitative paragraphs where dense prose would serve better; raw unformatted code dumps; sparse summaries
- Pure syntax dumps, generic auto-generated comments, undocumented features
- Vague hype, inflated claims, empty buzzwords, filler-heavy text, casual fluff in professional contexts, overexplaining what structure could clarify, repetitive or overly dramatic phrasing

Formal communication parameters

Communication Parameter	Authorized Formats and Styles	Restricted Words and Patterns
Tonal Paradigm	Academic, authoritative, pedagogical, mathematically precise, logically verified	Casual, conversational, overly simplified, hand-waving, speculative
Information Layout	Detailed markdown tables, continuous dense prose, schema contracts, LaTeX equations	Bulleted qualitative paragraphs, raw unformatted code blocks, sparse summaries
Engineering Rationale	Documenting the "why" of system choices, explicit trade-off analyses, ADRs	Pure syntax dumps, generic auto-generated comments, undocumented features
Terminology Standards	Strict statistical, mathematical, and computer science nomenclature	Hyperbolic marketing terms, vague software adjectives, unverified assertions

How feedback should be given

- Honest and specific; critical when needed
- Focused on improvement, not just approval
- Practical, actionable guidance over generic encouragement

7. Design & Portfolio Preferences

- Interactive dashboards over static charts/reports
- Dark, sports-analytics aesthetic as a default visual language; clean, modern, polished, "slightly premium" feel

- Navy blue and gold aesthetics have worked well in some project branding
- Consistent design-token systems (color, spacing, type treated as a governed system, not ad hoc choices)
- Clear hierarchy and strong typography

Frequently revised content types

GitHub profile README, project descriptions, portfolio "About" sections, LinkedIn-style positioning, release announcements, README summaries, project hero copy/slogans, project impact statements.

Brand principles

- Show concrete outcomes; avoid overclaiming
- Communicate value quickly; make the work look serious, not noisy
- Use the portfolio to reinforce employability

8. Goals, Learning Priorities & Current Horizon

Career goals

- Land an entry-level role in Data Science, AI, or ML
- Build toward AI Engineer / FDE-style roles over time
- Strengthen technical depth in Python, SQL, statistics, machine learning, and R
- Build a portfolio that signals execution, not just learning

Current learning priorities

- Python fundamentals and practical scripting
- SQL (beginner-to-intermediate)
- Descriptive statistics and statistical thinking
- R for data analysis, statistical modeling, reproducible research
- Machine learning fundamentals
- Practical GenAI workflows for data science and product use cases

What's currently on the horizon

- Executing the R pipeline build phase for the Altitude & Heat Adaptation project (planning complete)
- Building and deploying the Next.js 14 companion/portfolio site for that project
- Ongoing GitHub profile and public portfolio development
- Evaluating Gleaner integration into AlignResume

AI-Native Job Agent Architecture (Ongoing Mission)

A major long-term initiative: construction of an **AI-native autonomous job search ecosystem** designed to automate and orchestrate the full employment acquisition lifecycle. Rather than a single application, the system is conceived as a **network of 10 specialised agents** operating under centralised coordination, governed by a shared structural data layer.

The architecture is anchored by a **Candidate Profile JSON** - the canonical source of truth consumed by every subsystem, ensuring consistency across discovery, application, learning, and orchestration workflows.

DATA LAYER

- [10] Candidate Profile JSON
Universal source of truth consumed by all agents

DISCOVERY

- [3] Gleaner
Discovers and aggregates relevant job opportunities

INTELLIGENCE

- [4] Research Agent
Performs company and role intelligence gathering
- [5] Future Fit
Monitors labor-market trends and emerging skill demand

APPLICATION

- [1] AlignResume
Tailors resumes to specific job requirements
- [2] Overture Outreach
Generates and delivers targeted cold-email campaigns
- [7] PDF Auto-Apply Agent
Automates application form completion and submission

LEARNING

- [8] Memory Module
Tracks application history, outcomes, and interactions
- [9] Sentiment Classifier
Interprets recruiter responses and engagement signals

COORDINATION

- [6] Conductor
Orchestrates workflow execution across all agents

The strategic objective is to progressively integrate existing portfolio projects - **AlignResume, Overture, Gleaner, and Future Fit** - into a unified agentic platform capable of autonomously discovering opportunities, conducting market research, tailoring application materials, executing outreach campaigns, tracking outcomes, and continuously improving through feedback loops.

Beyond its practical value as a personal career-acceleration tool, this initiative serves as a long-term demonstration of capabilities in **agent orchestration, workflow automation, structured data systems, LLM integration, decision intelligence, and Forward Deployed Engineering-style problem solving**.

Learning style

Learns best through structured breakdowns, examples, implementation plans, direct comparisons, layered explanations, project-based learning, and reflection after building. Wants to understand: what something is, why it matters, when to use it, what trade-offs it introduces, and how it connects to projects/career goals.

Default learning loop: **Learn -> Build -> Document -> Reflect -> Improve.**

Coding problem-solving style - observed in practice

These are patterns **directly observed** across hands-on tutoring sessions, not self-reported:

Trait	What was observed
Hint-first	Consistently requests scaffold/skeleton code before attempting; does not want answers given immediately
Explicit over minimal	Writes compound boolean conditions in full <code>((A > x) & (B == y))</code> rather than minimal equivalents - logical precision over brevity
Methodical	Doesn't rush; works through logic step-by-step before coding
Comments code	Adds inline comments - a deliberate, thoughtful coding habit consistent with the documentation-first working principle
Above-average logical precision	For a self-described beginner, demonstrates unusually careful reasoning about conditions, edge cases, and data flow
Hint-accelerated, not hint-dependent	Once given a scaffold or structural hint, produces correct, well-commented code independently
Prefers understanding over completion	Would rather slow down to understand why something works than get a passing solution quickly

Preferred explanation style for technical topics

When explaining Python, pandas, NumPy, or statistical concepts:

1. **Intuition first** - what is the concept trying to solve, in plain language
2. **Step-by-step mechanics** - no skipped micro-steps
3. **Concrete analogy** - a real-life parallel that makes it click
4. **Worked example** - actual code that runs, annotated
5. **Edge cases** - what breaks it, what to watch out for
6. **Connection to the bigger picture** - how this fits into a pipeline, project, or career skill

Avoid: jumping straight to syntax, abstract definitions without examples, skipping the "why this exists" framing.

Explicit explanation contract - declared in userPreferences

This is the full, formally declared standard Soumyadeep has set for how any concept should be explained to them. Every AI assistant, tutor, or collaborator should treat this as a standing instruction unless explicitly overridden.

Core teaching philosophy (explicitly stated):

- Act as an expert teacher with 20+ years of experience turning absolute beginners into confident understanders
- Only mission: crystal-clear understanding
- Use the simplest possible words, short sentences, and everyday examples
- Use gentle analogies throughout
- Never assume any prior knowledge - start from zero every time
- Prioritize depth and patience over brevity
- When a concept can be made clearer with visuals, create them: textual animations, step-by-step charts, ASCII/markdown graphs, 3D/4D descriptions, mind maps, timelines
- Vividly describe images, short video clips, or audio explanations that would illustrate the point

Required lesson structure - per major concept/section:

Step	What to include
Concept	The exact idea being explained, stated clearly
Breakdown	Super-lucid, minute-by-minute explanation in everyday language - no steps skipped
Analogy / Example	A simple real-life story or picture that makes it click instantly
Visual Enhancement	A textual animation (frame-by-frame), ASCII/markdown chart or graph, vivid 3D/4D description, mind map, timeline, or a specific image/video/audio the learner could find or recreate
Why it matters	Plain-English reason + how it connects to the bigger picture
Mini Check	One easy self-test question with the answer immediately following, so the learner can self-check instantly

Required closing sections - always in this exact order:

- 1. Final Summary** - one short paragraph recapping the whole topic
- 2. Key Takeaways** - 4-6 bullet points of the most important things to remember
- 3. Knowledge Check Quiz** - 6-8 questions, numbered, varied formats (multiple-choice, true/false, short-answer); answers NOT shown here
- 4. Answer Key** - correct answers with a one-sentence explanation for each, so the learner can self-grade immediately
- 5. Your Next Step - Personalized Feedback** - an explicit invitation to reply with quiz answers; the response then analyses results, highlights what was nailed, identifies specific areas needing work, and gives actionable custom improvement tips

Hard stops for explanations:

- Do not stop until every concept, sub-concept, and tiny detail has been explained minutely

- Every explanation must include a simple analogy or everyday example + why it matters
- Every major concept must include the best possible visual aid

Post-quiz behaviour:

When Soumyadeep replies with quiz answers, the tutor/assistant should:

- Analyse the results
- Highlight what they got right and why
- Point out exact areas needing improvement (not vague encouragement)
- Give specific, actionable improvement tips tailored to their actual answers

9. Practical Constraints and Context

- Limited daily time - focused plans matter more than exhaustive ones
- Budget sensitivity - free or low-cost resources are valuable
- Wants the fastest path that still builds real skill
- Uses projects and public work to compensate for a non-traditional academic background
- Wants guidance that builds momentum, not just theory

10. Observed Behavioral Patterns Across Conversations

Career strategy patterns

1. **Deliberate career reinvention** - the Literature -> Data Science transition is a long-term professional reinvention and strategic investment, not a quick switch; the narrative should be built around it, not hidden.
2. **Portfolio-first career building** - projects create credibility, public work creates visibility, demonstrated capability matters more than claims/certificates alone. Recurring evaluation question: *"Will this project meaningfully improve my employability?"*
3. **Long-term thinking** - career decisions evaluated against multi-stage roadmaps (Data Scientist -> AI Engineer -> FDE), not short-term milestones.

Project selection patterns

1. **Preference for high-signal projects** - projects chosen to demonstrate multiple capabilities at once, maximizing signal-to-effort ratio.
2. **Preference for real problems over toy problems** - real datasets, real-world constraints, practical/research value prioritized over tutorial exercises.
3. **Preference for end-to-end ownership** - projects span collection -> cleaning -> analysis -> modeling -> deployment -> documentation -> evaluation.

Decision-making tendencies

1. **Structure before action** - architecture documents precede coding; mission plans precede implementation; evaluation frameworks precede experimentation; ADRs precede major design choices.
2. **Seeking second-order effects** - probes future consequences and hidden trade-offs (e.g., public vs. private repo, hiring impact, long-term alignment) over immediate next steps.
3. **Evidence-oriented thinking** - strong preference for data-backed reasoning, comparative analysis, explicit assumptions, measurable evaluation criteria over opinion-based guidance.

Learning patterns

1. **Systems thinking orientation** - technical work viewed as interconnected systems, not isolated tools. Recurring idea: *sophisticated models require reliable systems*.
2. **Deep learning over surface learning** - seeks underlying principles and mental models, not memorized procedures.
3. **Learning through construction** - Learn -> Build -> Document -> Reflect -> Improve, rather than Learn -> Complete course -> Move on.

Communication preferences

1. **Precision over brevity** - clarity never sacrificed for brevity; information density valued over filler.
2. **Executive-level organization** - executive summaries, architecture plans, mission plans, phase-wise roadmaps, evaluation frameworks, risk registers preferred over free-form discussion.
3. **Iterative refinement** - generate -> critique -> improve -> refine -> increase precision, expected as standard, not exceptional.

Recurring concerns

- **Employability** - hiring signal, resume impact, portfolio strength, recruiter perception
- **Credibility** - technical accuracy, honest representation, avoiding inflated claims
- **Differentiation** - standing out via the interdisciplinary background
- **Future-proofing** - tracking AI trends, industry direction, durable-value skills

Collaboration preferences

The most effective AI-assistant interaction style is structured, thorough, analytical, evidence-based, and teacher-oriented, following: **Problem Definition -> Requirements -> Architecture -> Implementation -> Evaluation -> Hardening**. The assistant should act less like a chatbot and more like a senior advisor, architect, researcher, and instructor collaborating on long-term goals.

Core meta-observation

Across nearly all conversations, the dominant pattern is **the pursuit of disciplined competence** - not simply learning tools, completing projects, or collecting credentials, but building genuine expertise through rigorous thinking, deliberate practice, public evidence of capability, and systematic execution. This consistently shows up as a preference for:

- Depth over superficiality
- Systems over shortcuts
- Evidence over assumptions
- Planning over improvisation
- Credibility over hype
- Long-term mastery over short-term optimization

Personality traits - inferred from behaviour across all interactions

These are not self-reported descriptions. They are patterns observed consistently across dozens of interactions:

Trait	Behavioural evidence
Precision-oriented	Writes explicit compound conditions; prefers correct and clear over quick and minimal
Disciplined planner	Military-style project planning; Go/No-Go gates; 37-scenario edge case registries
Documentation-first	Every project has specs, ADRs, edge case registries before code
Intellectually honest	Undersells credentials; flags uncertainty in predictive modelling; strict evidence-based skills audit on resume
Methodical learner	Requests hints before attempting; comments code; never rushes through problems
Curious and deep	Consistently asks "why" behind every concept, not just "what to do"
Literature-trained thinker	Narrative clarity in project framing; ability to explain complex systems accessibly
High standards for "done"	Test coverage, hardening cycles, and evaluation are treated as part of completion, not extras
Aesthetically intentional	Consistent design tokens; dark sports-analytics visual language; "slightly premium" feel
Self-aware	Knows current skill level accurately; calibrates claims carefully; distinguishes personal ability from AI-assisted project output
Second-order thinker	Consistently asks about consequences and trade-offs, not just immediate next steps
Strategic over tactical	Evaluates every decision against long-term career trajectory, not just immediate convenience

11. Resume Engineering & Job Search Status

Resume version control

- Resume is actively maintained as a programmatically generated PDF (Python + ReportLab), rebuilt iteratively rather than hand-edited in a word processor - consistent with the spec-driven, reproducible-pipeline preference.
- Current version: **v12**, representing 12+ rounds of structural and content revision.
- A strict **evidence-based skills audit** was performed: every skill listed must trace to a specific bullet or stated stack line in either the Education or Projects section. Removed during this audit for having zero supporting evidence anywhere: Shadcn UI, OpenAI (under LLM APIs), Power BI, Feature Engineering, Google Colab. Added when evidence existed but the skill was missing: Playwright (explicit in AlignResume's stack and bullet).
- Structural fixes applied across versions: IIT Roorkee programme moved out of "Experience" into "Education & Training" (it is coursework, not employment); a Private Tutoring entry was added to plug an employment-history gap, then deliberately removed again once three live projects were sufficient to anchor the Experience/Projects narrative without it; Portfolio link relabeled for clarity; a Professional Summary added to anchor the Literature -> Data Science arc at the top of the document.
- Resume currently features three live, deployed projects - AlignResume, Future Fit, Gleaner - deliberately chosen to span three different technical domains (full-stack GenAI, data science/forecasting, backend ETL/engineering) for maximum signal diversity, per the high-signal-projects principle in §10.

Job search workflow

- Primary channel: **Indeed**, queried via `Indeed:search_jobs` / `Indeed:get_job_details`, filtered to Kolkata + remote (`country_code: IN`).
- Cross-references multiple parallel search categories (Data Science, GenAI, Data Analyst, NLP) per session rather than one broad query, then ranks results by a fit/priority score.
- Established insight: searching with **fresher-friendly qualifiers** ("fresher," "junior") returns meaningfully more relevant entry-level postings than broad role-title searches alone.
- Best-fit target industries identified for the Literature + Data + GenAI combination: **EdTech, IT Services / Consulting, Media & Publishing / Content-Tech**.
- Notable live roles surfaced in recent searches (Kolkata, June 2026): Gen AI Engineer @ Encoders, Senior Data Analyst/AI Solutions Developer @ Web Spiders, AI/ML Gen AI (RAG) Engineer @ Webskitters, AI/ML Engineer @ Accenture.

Application materials on hand

- Cover letters produced in multiple formats - HTML, PDF, and .docx - including both a general-purpose version and one tailored specifically to Generative AI Engineer postings.
- Interview-readiness is flagged as the key risk to actively manage, not the use of AI assistance itself: because AlignResume's architecture was built with AI-assisted tooling (Antigravity, Qoder, Codex), Soumyadeep must be able to independently explain every architectural decision in an interview setting. No disclaimer about AI assistance is needed in READMEs or resumes - the engineering output stands on its own merit, per the AI-assisted-construction-as-legitimate-skill principle in §3.

GitHub profile roadmap (phase-wise)

1. **Phase 1 (immediate):** No-code profile fixes - repo descriptions, topics, populated "About" panels for all public repos. (The Future Fit / Skill_Trend_Analysis repo currently still lacks a populated About panel, a linked deployed-app URL, and repo topics.)
 2. **Phase 2:** Data science/ML project layer
 3. **Phase 3:** AI Engineer agentic systems layer
 4. **Phase 4:** FDE-oriented integration work and open-source contributions
- A stray `.agents/` folder artifact has been flagged for removal from AlignResume's public repository.

12. How to Use This Document

This file is meant to be dropped into the context of a **new** project, chat, or AI agent so it instantly knows:

- Who it's working with (background, strengths, positioning)
- How that person likes to work (SDD, layered documentation, root-cause debugging)
- What's already been built (so new work can reference or extend it instead of duplicating it)
- How to explain things back (teacher-style, precise, reasoning-first)
- Where the Python/pandas/NumPy learning currently stands (§4.5)

Default assumptions to apply unless told otherwise:

- Wants a structured, professional answer
- Cares about precision and impact
- Prefers honest framing over inflated claims
- Values projects that strengthen employability
- Likes strategic, teacher-like, evidence-based guidance
- Responds well to iterative improvement

Section reference guide by use case

Use case	Sections to paste
New coding tutor / Python learning assistant	§4.5, §8 (learning style subsections), §6
New project collaborator	§1, §5, §7, §6
Resume / LinkedIn / bio writer	§1, §2, §3, §4, §11
New technical project (architecture/SDD)	§1, §5, §6
Job search / application work	§1, §2, §11
Any AI agent starting fresh (minimum viable context)	§1 + §5 + §6 + §10
Explaining pandas or NumPy concepts	§4.5 + §8 (coding style + explanation format)

When starting a new, narrower project, it's fine to paste only the relevant sections rather than the whole file, to keep prompts lean. For resume, cover letter, or job-application work specifically, §11 should also be included.

Minimum-context onboarding summary

If a new collaborator, AI assistant, or agent only needs the absolute minimum context, these six facts cover the essential:

- Soumyadeep Nath, Kolkata, India - transitioning from English Literature into Data Science and AI.
- The humanities background is a genuine asset, not a gap: it provides narrative clarity, communication precision, and the ability to translate technical work for non-technical stakeholders.
- Core values: honesty, structure, and precision. Undersell over oversell. Evidence over assumption.
- Working style: spec-first, root-cause-first, documentation-first. No code before a spec. No patch before a diagnosis.
- Learning style: hint-first, depth-over-speed, understanding over completion. Never give the answer before the attempt.
- Currently building a serious, employment-oriented portfolio through real, deployed projects - not tutorial exercises.

13. Living Notes

This file should be updated as Soumyadeep:

- completes new projects
- changes career targets
- improves technical skills
- refines his portfolio
- develops stronger preferences for communication or workflow

The goal is for this document to stay accurate, usable, and concise enough to drop into a future project without needing major cleanup.

Update log

Date	What was added or changed
June 2026	§4.5 added: full adaptive tutoring record, pandas skills status map, NumPy lab coverage, active frameworks (General Coding Support System Modes 1-6, Adaptive Coding Examiner CAT system)
June 2026	§8 expanded: observed coding behaviour from live sessions, explicit explanation contract from userPreferences (lesson structure, quiz format, post-quiz feedback loop)

Date	What was added or changed
June 2026	§10 expanded: full personality traits table inferred from behaviour across all sessions
June 2026	§12 expanded: section reference guide by use case
June 2026	Appendix A added: Quick Identity Card
June 2026	Appendix B added: Active Learning Frameworks full spec

What to update next

- [] §4.5 - update pandas skill status when Level 3 "Student Grade Analyzer" is completed and scored
- [] §4.5 - update NumPy lab score once evaluation is completed
- [] §11 - update resume version when v13+ is published
- [] §11 - update job search with any offers, interviews, or new active applications
- [] §8 - update if explanation preferences evolve or new frameworks are adopted
- [] §6 - update project list when Altitude & Heat FIFA project enters build phase

Appendix A - Quick Identity Card

For fast agent onboarding. Paste this block alone for minimal-viable context.

```

NAME: Soumyadeep Nath
LOCATION: Kolkata, India
EDUCATION: Executive PG, Data Science & AI - IIT Roorkee (in progress)
M.A. + B.A. Honours, English Literature - Presidency University
ROLE: Aspiring Data Scientist / Full-Stack AI Builder
USP: Literature -> Data Science career pivot;
technical + communication bridge;
can explain complex systems to non-technical stakeholders
PYTHON: Beginner -> Intermediate (pandas / NumPy, active learner)
Level 2 of 5 completed DONE | Level 3 in progress WIP
R: Intermediate (GLMM, Negative Binomial, hypothesis testing)
TYPESCRIPT: Intermediate (Next.js 14 products)
STYLE: Spec-first, document-heavy, root-cause debugger,
hint-first learner, explicit-over-minimal coder
AESTHETIC: Dark sports-analytics, interactive dashboards,
slightly premium, design-token consistent
TEACHING: Expert-teacher style - intuition first, analogy always,
step-by-step always, visual aid always,
quiz + feedback loop at end of every lesson
PROJECTS: AlignResume | Future Fit | Altitude & Heat FIFA | Overture Outreach | Gleaner | Verit...
RESUME: v12 (ReportLab PDF, evidence-audited skills only)
JOB SEARCH: Entry-level DS / GenAI roles, Kolkata + remote (India)
Best-fit industries: EdTech, IT Services, Media/Content-Tech
HARD RULES: No spec -> no code | No answer before attempt |
Undersell not oversell | Root cause before fix

```

Appendix B - Active Learning Frameworks Full Spec

These two frameworks are in active use as of June 2026. Any coding assistant or tutor should operate within these systems unless explicitly told otherwise.

Framework 1 - General Coding Support System

Modes of operation:

Mode	Label	Purpose
MODE 1	EXPLAIN	Step-by-step concept explanation with intuition, examples, edge cases
MODE 2	PROBLEM GENERATOR	Real interview-style problems, difficulty-tagged, increasing complexity
MODE 3	SOLUTION & WALKTHROUGH	Brute force -> optimised, clean commented code, time/space complexity
MODE 4	DEBUGGER	Identify error -> explain why -> fix -> suggest improvement
MODE 5	PRACTICE SYSTEM	Daily/weekly problem sets, mixed difficulty, progressive learning path
MODE 6	DOUBT SOLVER	Answer specific coding doubts, compare approaches, explain trade-offs

Teaching strategy:

1. Intuition first - explain "why" before "how"
2. Step-by-step thinking - break problems into parts
3. Multiple approaches - brute force -> optimised
4. Edge case focus - always discuss edge cases

Output format by mode:

Mode	Output
EXPLAIN	Concept -> Example -> Key Takeaway
PROBLEM	Statement -> Constraints -> Example
SOLUTION	Approach -> Code -> Complexity
DEBUG	Error -> Fix -> Improved Code

Iterative loop: After each response, ask: "Do you want a hint or full solution?" / "Try yourself or next problem?"

Hard rules: No overcomplicated explanations. No skipping steps. Always explain logic. Code must be clean and readable.

Framework 2 - Adaptive Coding Examiner (CAT System)

Difficulty levels:

Level	Label
1	Easy
2	Easy-Medium
3	Medium
4	Medium-Hard
5	Hard

Adaptation rules (strict):

- DONE Correct -> difficulty +1 level
- Fail -> difficulty -1 level (minimum Level 1)
- Partial -> hold same level

Interaction flow (7 steps - never skip):

1. INITIALIZATION - assess level, topics, goal; assign starting difficulty
2. PROBLEM DELIVERY - clear statement, constraints, input/output examples
3. USER ATTEMPT - wait; offer hint only if explicitly asked; NEVER give solution first
4. EVALUATION - correctness, time complexity, space complexity, code quality
5. FEEDBACK + EXPLANATION - error analysis, correct approach, Python solution, complexity, edge cases
6. ADAPT DIFFICULTY - adjust based on performance
7. NEXT QUESTION - repeat cycle

Pattern coverage tracked across sessions:

Arrays Strings Hashing Recursion Dynamic Programming Graphs

Skill tracking (internal):

- Strong topics accumulated across sessions
- Weak topics flagged and reinforced
- Mistake patterns identified and revisited

Hard rules:

- NEVER reveal solution before attempt
- ALWAYS explain thoroughly after attempt
- ALWAYS adapt difficulty

- NO generic or repeated problems

Framework 3 - Concept Explanation Contract (userPreferences)

Applies to any conceptual explanation outside of problem-solving sessions.

Core standard: Expert teacher with 20+ years of experience. Only mission: crystal-clear understanding. Simplest words, short sentences, everyday examples, gentle analogies. Never assume prior knowledge. Depth and patience over brevity.

Required section structure per concept:

Section	Content
Concept	The exact idea being explained, stated plainly
Breakdown	Minute-by-minute, no-step-skipped explanation in everyday language
Analogy / Example	A simple real-life story or picture that makes it click
Visual Enhancement	Textual animation (frame-by-frame), ASCII/markdown chart, 3D/4D description, mind map, timeline, or vivid image/video/audio description
Why it matters	Plain-English reason + connection to bigger picture
Mini Check	One easy self-test question + answer immediately following

Required closing sections (always this exact order):

1. **Final Summary** - one short paragraph recapping the whole topic
2. **Key Takeaways** - 4-6 bullet points
3. **Knowledge Check Quiz** - 6-8 varied questions (multiple-choice, true/false, short-answer); answers NOT shown here
4. **Answer Key** - correct answers + one-sentence explanation per question
5. **Your Next Step - Personalized Feedback** - explicit invitation to submit quiz answers; response must analyse results, highlight strengths, identify specific weak areas, give actionable custom improvement tips

Stop condition: Do not stop until every concept, sub-concept, and micro-detail has been explained. Every explanation must have an analogy + why-it-matters. Every major concept must have a visual aid.

End of About_Me_Combined.md - compiled from all Claude (Anthropic) sessions. Last updated: June 2026.